

Appendix B

Cam controller

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Camera_Controller : MonoBehaviour
{
    [Header("Settings")]
    // antenna cam type style
    public Transform car;
    public float distance = 6.4f;
    public float height = 1.4f;

    [Header("Damping")]
    public float Rotation_Damping = 3.0f;
    public float Height_Damping = 2.0f;

    [Header("Zoom Settings")]
    public float Zoom_Ratio = 0.5f;
    public float FOV = 60f;
    // 60<FOV<80 otherwise screen bug
    private Vector3 rotationVector;

    void LateUpdate() {
        float wantedAngle = rotationVector.y; //maf
        float wantedHeight = car.position.y + height;
        float myAngle = transform.eulerAngles.y;
        float myHeight = transform.position.y;

        myAngle = Mathf.LerpAngle(myAngle, wantedAngle,
Rotation_Damping*Time.deltaTime); //more maf
        myHeight = Mathf.Lerp(myHeight, wantedHeight,
Height_Damping*Time.deltaTime);

        Quaternion currentRotation = Quaternion.Euler(0, myAngle, 0);
        transform.position = car.position;
```

```

        transform.position -= currentRotation *
Vector3.forward*distance;

        Vector3 temp = transform.position; //temporary variable so
Unity doesn't complain
        temp.y = myHeight;
        transform.position = temp;
        transform.LookAt(car);
    }

    void FixedUpdate(){
        Vector3 localVelocity =
car.InverseTransformDirection(car.GetComponent<Rigidbody>().velocity);
        if (localVelocity.z < -0.1f){
            Vector3 temp = rotationVector; // temporary variables seem
to be removed after a closing bracket "}" we can use the same variable
name multiple times!!!!!!!!!!!!!!!!!!!!!!
            temp.y = car.eulerAngles.y + 180;
            rotationVector = temp;
        }

        else{
            Vector3 temp = rotationVector;
            temp.y = car.eulerAngles.y;
            rotationVector = temp;
        }

        //Setting the field of view of the camera again(ig):
        float acc = car.GetComponent<Rigidbody>().velocity.magnitude;
        GetComponent<Camera>().fieldOfView = FOV + acc * Zoom_Ratio *
Time.deltaTime; //h removd * Time.deltaTime but it works better if you
leave it like this ig bc unity complains
    }
}

```

Car controller

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

```

```

using UnityEngine.SceneManagement;
using UnityEngine.UI;
using TMPro;
using System.Linq;
// note: audio sources will act as placeholders due to audio laws and
rules ;(
public class Car_Controller : MonoBehaviour
{
    //Public Variables
    [Header("Wheel Colliders")]
    public List<WheelCollider> Front_Wheels; //The front wheels
    public List<WheelCollider> Back_Wheels; //The rear wheels

    [Header("Wheel Transforms")]
    public List<Transform> Front_Wheel_Transforms; //The front wheel
transforms
    public List<Transform> Back_Wheel_Transforms; //The rear wheel
transforms

    [Header("Wheel Transforms Rotations")]
    public List<Vector3> Front_Wheel_Rotation; //The front wheel
rotation Vectors
    public List<Vector3> Back_Wheel_Rotation; //The rear wheel rotation
Vectors

    [Header("Car Settings")]
    public float Motor_Torque = 400; //Motor torque for the car
    public float Max_Steer_Angle = 25f; //The Maximum Steer Angle for
the front wheels
    public float BrakeForce = 150f; //The brake force of the wheels
    public float Maximum_Speed; //The top speed of the car

    [Space(15)]

    public float handBrakeFrictionMultiplier = 2; //The handbrake
friction multiplier
    private float handBrakeFriction = 0.05f; //The handbrake friction
    public float tempo; //Tempo (don't edit this)

    [Header("Boost Settings")]
    public bool enable_boost; //Use boost bool
    public float Boost_Cooldown = 10f; //Boost cooldown time (keep low)
    public float Boost_Amount = 10f; //Boost amount (10<x recommended)

```

```

public KeyCode Boost_KeyCode; //Key for boost

[Space(15)]

public bool Enable_Boost_particles; //Use boost particles?
public ParticleSystem[] Boost_particles; //Boost particles
list/array

[Header("Car States")]
public bool Use_Car_States; //Use car states?
public bool Car_Started; //Car started?
public KeyCode Car_Start_Key; // start the car
public KeyCode Car_Off_Key; // turn car off

[Header("Audio Settings")]
public bool Enable_Audio; //Use audio? (copyright + complex ...
leave for future use?)
public bool Enable_Engine_Audio; //use engine audio? (looped, for,
restrictions or passive?)
public AudioSource Engine_Sound; //Audio source for engine sound
public float Minimum_Pitch_Value; //Minimum pitch value for engine
public float Maximum_Pitch_Value; //Maximum pitch value for engine

[Space(15)]

public bool Enable_Horn; //Use horn?
public AudioSource Horn_Source; //Audio source for horn
public KeyCode Car_Horn_Key; //Key to use horn

[Header("Crash System")]
public bool Enable_Crash_Noise; //Use crash sounds?
public string[] Crash_Object_Tags; //Objects that will cause car to
make the crash sound (tires barriers buildings)
public AudioSource Crash_Sound; //Audiosource for the car crash
sounds

[Header("Drift Settings")]
public bool Set_Drift_Settings_Automatically = true; //Set the
drift setting automatically?
public float Forward_Extremium_Value_When_Drifting; //Forward
(oversteer>) value when drifting
public float Sideways_Extremium_Value_When_Drifting; //Sideways
(oversteer>) value when drifting

```

```

[Header("Light Setting(s)")]

[Header("Light Settings (With Light Objects)")]
public bool Enable_Headlights_Lights; //Enable headlights? (These
are light objects) ( use probe or smth)
public bool Enable_Brakelights_Lights; //Enable brakelights? (These
are light objects)
public bool Enable_Reverselights_Lights; //Enable reverse lights?
(These are light objects)
public KeyCode Headlights_Key; //Key to turn on headlight(s)


public Light[] HeadLights; //Headlight object(s) list/array
public Light[] BrakeLights; //Brakelight object(s) list/array
public Light[] ReverseLights; //Reverse light object(s) list/array


[Space(15)]

[Header("Light Settings (With MeshRenderers)")]
//dont have to use just use probe ig
public bool Enable_Headlights_MeshRenderers; //Enable Headlight(s)?
(These are meshrenders)
public bool Enable_Brakelights_MeshRenderers; //Enable
Brakelight(s)? (These are meshrenders)
public bool Enable_Reverselights_MeshRenderers; //Enable Reverse
light(s)? (These are meshrenders)


public MeshRenderer[] HeadLights_MeshRenderers; //List/array for
the headlight meshrendere(s)
public MeshRenderer[] BrakeLights_MeshRenderers; //List/array for
the brakelight meshrendere(s)
public MeshRenderer[] ReverseLights_MeshRenderers; //List/array for
the reverse light meshrendere(s)


[Header("Particle System(s) Settings")]
public bool Use_Particle_Systems; //Use the particle system(s)
public ParticleSystem[] Car_Smoke_From_Silencer; // . . .


[Header("Scene Settings")]
public bool Use_Scene_Settings; //Use scene setting(s)
public KeyCode Scene_Reset_Key = KeyCode.R; //Scene reset key

```

```

[Header("Other Settings")]
public Transform Center_of_Mass; //Centre of mass of car (empty
object )
public float frictionMultiplier = 3f; //Friction Multiplier
public Rigidbody Car_Rigidbody; //Car rigidbody

[Header("Debug Values")]
public float Car_Speed_KPH; //The car speed in KPH (use this and be
normal)
public float Car_Speed_MPH; //The car speed in MPH

[Space(15)]

public bool HeadLights_On; //Headlights on/off?

//Debug Values in Int Form
public int Car_Speed_In_KPH; //Car speed in KPH (integer form)
public int Car_Speed_In_MPH; //Car speed in MPH (integer form)

public bool Is_Flying () //bool for if the car is flying or not
{
    if (!Back_Wheels[0].isGrounded && !Front_Wheels[0].isGrounded)
{
        return true;
    } else
        return false;
}

//private Variables
private Rigidbody rb; //The rb
private float Brakes = 0f; //Brakes
private WheelFrictionCurve Wheel_forwardFriction,
Wheel_sidewaysFriction; //Wheel friction curve(s)
private float Next_Boost_Time; //Next boost time

//Private Audio Variables(dont mess with)
private float pitch; //Pitch

//hidden Variables (not private but hidden in inspector)
[HideInInspector] public float currSpeed; //Current speed

void Start(){
    //To Prevent The Car From Toppling When Turning Too Much

```

```

        rb = GetComponent<Rigidbody>(); //get rigidbody
        rb.centerOfMass = Center_of_Mass.localPosition; //Set the
centre of mass of the rigid body to the centre of mass transform

        //Play Car Smoke Particle System
        if(Use_Particle_Systems){
            foreach(ParticleSystem P in Car_Smoke_From_Silencer){
                P.Play(); //Play the smoke from silencer particle
system
            }
        }

        //Here we just set the lights to turn on and off at play.

        //We turn the headlights on/off here
        Turn_Off_Headlights();
        Turn_On_Headlights();

        //Here we turn the reverse light(s) off
        if(Enable_Reverselights_Lights){
            foreach(Light R in ReverseLights){
                R.enabled = false;
            }
        }

        if(Enable_Reverselights_MeshRenderers){
            foreach(MeshRenderer RM in ReverseLights_MeshRenderers){
                RM.enabled = false;
            }
        }

        //Here we turn off the brakelights
        Turn_Off_Brakelights();

        //Turning some things off if their options are disabled
        // dont use
        if(!Enable_Horn && Horn_Source != null){
            Horn_Source.gameObject.SetActive(false); //is horn is not
enabled and the horn source there, disable the horn
        }

        if(!Enable_Engine_Audio && Engine_Sound != null){

```

```

        Engine_Sound.gameObject.SetActive(false); //Disable the
engine sound if the engine sound has not been enabled and it is set to
some audio source.
    }

    if(!Enable_Audio && (Engine_Sound != null || Horn_Source !=
null)){
        Horn_Source.gameObject.SetActive(false);
        Engine_Sound.gameObject.SetActive(false);
    }
}

public void FixedUpdate(){
    //Turning car off
    if(Input.GetKeyDown(Car_Off_Key) && (Car_Speed_KPH >= 0 &&
Car_Speed_KPH <= 1.5f) && Use_Car_States){ //if the car off key has
been pressed and the car speed is 0 and the "use car states" is true
        Turn_Off_Car(); //Turn car off
    }

    //Turning Car on
    if(Input.GetKeyDown(Car_Start_Key) && Use_Car_States){ //if the
"use car states" is true and that the car start key is pressed
        Car_Started = true;
    }

    //If the car states are not in use
    if(!Use_Car_States){
        Car_Started = true;
    }

    //Check the keys for headlights and turn them off/on
    if(Input.GetKeyDown(Headlights_Key) && Car_Started == true){
//if the headlights key was pressed
        if(!HeadLights_On){
            HeadLights_On = true; //set the headlights on to true
            Turn_On_Headlights(); //turn on headlights
        }

        else{
            HeadLights_On = false; //Set the headlights on to false
            Turn_Off_Headlights(); //turn off the headlights
        }
    }
}

```

```

    }

    if(Car_Started == false){ //if the car is off
        Turn_Off_Headlights(); //turn the headlights off
    }

    //Applying Maximum Speed
    if(Car_Speed_In_KPH < Maximum_Speed && Car_Started){ //if the
car's current speed is less than the maximum speed
        //Let car move forward and backward
        foreach(WheelCollider Wheel in Back_Wheels){
            Wheel.motorTorque = Input.GetAxis("Vertical") *
((Motor_Torque * 5)/(Back_Wheels.Count + Front_Wheels.Count));
        }
    }

    if(Car_Speed_In_KPH > Maximum_Speed && Car_Started){ //if the
car's current speed is more than the top speed
        //Don't let the car accelerate anymore so it does not
exceed the maximum speed
        foreach(WheelCollider Wheel in Back_Wheels){
            Wheel.motorTorque = 0;
        }
    }

    //Making The Car Turn/Steer
    if(Car_Started){
        foreach(WheelCollider Wheel in Front_Wheels){
            Wheel.steerAngle = Input.GetAxis("Horizontal") *
Max_Steer_Angle; //Turn the wheels
        }
    }

    //Changing speed of the car
    Car_Speed_KPH = Car_Rigidbody.velocity.magnitude * 3.6f;
//Calculate car speed in KPH
    Car_Speed_MPH = Car_Rigidbody.velocity.magnitude * 2.237f;
//Calculate the car's speed in MPH

    Car_Speed_In_KPH = (int) Car_Speed_KPH; //Convert the float
values of the speed to int
    Car_Speed_In_MPH = (int) Car_Speed_MPH; //Convert the float
values of the speed to int

```

```

        //Make Car Boost
        if(Input.GetKeyDown(Boost_KeyCode) && Car_Started &&
Next_Boost_Time < Time.time){
            //BOOST CAR
            Boost_Function();
            Next_Boost_Time = Time.time + Boost_Cooldown; //The
cooldown for the car
        }

        //Make Car Drift
        WheelHit wheelHit;

        foreach(WheelCollider Wheel in Back_Wheels){
            Wheel.GetGroundHit(out wheelHit);

            if(wheelHit.sidewaysSlip < 0 )
                tempo = (1 + -Input.GetAxis("Horizontal")) *
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

            if(tempo < 0.5) tempo = 0.5f;

            if(wheelHit.sidewaysSlip > 0 )
                tempo = (1 + Input.GetAxis("Horizontal") )*
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

            if(tempo < 0.5) tempo = 0.5f;

            if(wheelHit.sidewaysSlip > .99f || wheelHit.sidewaysSlip <
-.99f){
                //handBrakeFriction = tempo * 3;
                float velocity = 0;
                handBrakeFriction =
Mathf.SmoothDamp(handBrakeFriction,tempo* 3,ref velocity ,0.1f *
Time.deltaTime);
            }

            else{
                handBrakeFriction = tempo;
            }
        }

        foreach(WheelCollider Wheel in Front_Wheels){

```

```

Wheel.GetGroundHit(out wheelHit);

    if(wheelHit.sidewaysSlip < 0 )
        tempo = (1 + -Input.GetAxis("Horizontal")) *
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

        if(tempo < 0.5) tempo = 0.5f;

    if(wheelHit.sidewaysSlip > 0 )
        tempo = (1 + Input.GetAxis("Horizontal") ) *
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

        if(tempo < 0.5) tempo = 0.5f;

    if(wheelHit.sidewaysSlip > .99f || wheelHit.sidewaysSlip <
-.99f){
        //handBrakeFriction = tempo * 3;
        float velocity = 0;
        handBrakeFriction =
Mathf.SmoothDamp(handBrakeFriction,tempo* 3,ref velocity ,0.1f *
Time.deltaTime);
    }

    else{
        handBrakeFriction = tempo;
    }
}

    if((Input.GetAxis("Vertical") < 0) && Car_Started){ //Turn on
the reverse lights when car is reversing
        //Turn on reverse light(s)
        Turn_On_ReverseLights();
    }

    if((Input.GetAxis("Vertical") > 0) && Car_Started){
        //Turn off reverse light(s)
        Turn_Off_ReverseLights();
    }
}

public void Update(){
    //Scene Settings
    if(Use_Scene_Settings){

```

```

        if (Input.GetKeyDown(Scene_Reset_Key)) { //When the reset key
is pressed

SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
//Restart the current scene
        }
    }

    //Rotating The Wheels Meshes so they have the same position and
rotation as the wheel colliders
    var pos = Vector3.zero; //position value (temporary)
    var rot = Quaternion.identity; //rotation value (temporary)

    for (int i = 0; i < (Back_Wheels.Count); i++)
    {
        Back_Wheels[i].GetWorldPose(out pos, out rot); //get the
world rotation & position of the wheel colliders
        Back_Wheel_Transforms[i].position = pos; //Set the wheel
transform positions to the wheel collider positions
        Back_Wheel_Transforms[i].rotation = rot *
Quaternion.Euler(Back_Wheel_Rotation[i]); //Rotate the wheel transforms
to the rotation of the wheel collider(s) and the rotation offset
    }

    for (int i = 0; i < (Front_Wheels.Count); i++)
    {
        Front_Wheels[i].GetWorldPose(out pos, out rot); //get the
world rotation & position of the wheel colliders
        Front_Wheel_Transforms[i].position = pos; //Set the wheel
transform positions to the wheel collider positions
        Front_Wheel_Transforms[i].rotation = rot *
Quaternion.Euler(Front_Wheel_Rotation[i]); //Rotate the wheel
transforms to the rotation of the wheel collider(s) and the rotation
offset
    }

    //Make Car Brake w/o locking happening due to ABS being a thing
ig

    if (Input.GetKey(KeyCode.Space) && Car_Started) {
        Brakes = BrakeForce;

        Turn_On_Brakelights();
    }

```

```

        //Drifting and changing wheel collider values
        if(Set_Drift_Settings_Automatically){
            foreach(WheelCollider Wheel in Back_Wheels){
                Wheel_forwardFriction = Wheel.forwardFriction;
                Wheel_sidewaysFriction = Wheel.sidewaysFriction;

                Wheel_forwardFriction.extremumValue =
Wheel_forwardFriction.asymptoteValue = ((currSpeed *
frictionMultiplier) / 300) + 1;
                Wheel_sidewaysFriction.extremumValue =
Wheel_sidewaysFriction.asymptoteValue = ((currSpeed *
frictionMultiplier) / 300) + 1;
            }

            foreach(WheelCollider Wheel in Front_Wheels){
                Wheel_forwardFriction = Wheel.forwardFriction;
                Wheel_sidewaysFriction = Wheel.sidewaysFriction;

                Wheel_forwardFriction.extremumValue =
Wheel_forwardFriction.asymptoteValue = ((currSpeed *
frictionMultiplier) / 300) + 1;
                Wheel_sidewaysFriction.extremumValue =
Wheel_sidewaysFriction.asymptoteValue = ((currSpeed *
frictionMultiplier) / 300) + 1;
            }
        }

        if(!Set_Drift_Settings_Automatically){
            foreach(WheelCollider Wheel in Back_Wheels){
                //Variables getting assigned
                Wheel_forwardFriction = Wheel.forwardFriction;
                Wheel_sidewaysFriction = Wheel.sidewaysFriction;

                //Setting The Extremium values to the ones that the
user defined
                Wheel_forwardFriction.extremumValue =
Forward_Extremium_Value_When_Drifting;
                Wheel_sidewaysFriction.extremumValue =
Sideways_Extremium_Value_When_Drifting;
            }

            foreach(WheelCollider Wheel in Front_Wheels){
                //Variables getting assigned

```

```

        Wheel_forwardFriction = Wheel.forwardFriction;
        Wheel_sidewaysFriction = Wheel.sidewaysFriction;

        //Setting The Extrmum values to the ones that the
user(me) defined
        Wheel_forwardFriction.extremumValue =
Forward_Extremum_Value_When_Drifting;
        Wheel_sidewaysFriction.extremumValue =
Sideways_Extremum_Value_When_Drifting;
    }
}

else{
    Brakes = 0f;
}

//Apply brake force & no bb bc stock car
foreach(WheelCollider Wheel in Front_Wheels){
    Wheel.brakeTorque = Brakes; //set the brake torque of the
wheels to the brake torque
}

foreach(WheelCollider Wheel in Back_Wheels){
    Wheel.brakeTorque = Brakes; //set the brake torque of the
wheels to the brake torque
}

//Turn the brakelights on
if(!Input.GetKey(KeyCode.Space) && Car_Started){ //When the car
brake button is pressed
    Turn_Off_Brakelights();
}

//Audio System (Ph)
if(Enable_Audio){
    if(Enable_Engine_Audio && Car_Started){
        //Setting the pitch according to the speed of the car.
pitch = Car_Speed_In_KPH/Maximum_Speed + 1f;

        //Do this if the pitch variable exceeds the maximum
pitch value
        if(pitch > Maximum_Pitch_Value){

```

```

        pitch = Maximum_Pitch_Value;
    }

    //Do this if the pitch variable is lower than the
minimum pitch value
    else if(pitch < Minimum_Pitch_Value){
        pitch = Minimum_Pitch_Value;
    }

    //This actually sets the audio source pitch
surprisingly
    Engine_Sound.pitch = pitch;
}

if(Enable_Engine_Audio && !Car_Started){
    //Stop Engine
    Engine_Sound.Stop();
}

//Car Horn
if(Enable_Horn){
    if(Input.GetKey(Car_Horn_Key) &&
!Horn_Source.isPlaying){
        //Play the sound
        Horn_Source.Play();
    }

    if(!Input.GetKey(Car_Horn_Key)){
        //Stop playing the sound
        Horn_Source.Stop();
    }
}
}

void OnCollisionEnter(Collision col){
    //Play the crash sound when car crashes into an object with the
tag in the "Crash_Object_Tags" list
    if(Enable_Crash_Noise && Enable_Audio){
        foreach (string tag in Crash_Object_Tags){
            if(col.gameObject.tag == tag){
                //Play the crash sound:
                Crash_Sound.Play();
            }
        }
    }
}

```

```

    }

    else{
        //Stop playing the crash sound
        Crash_Sound.Stop();
    }
}

}

}

//Functions to turn on/off the brakelights

public void Turn_On_Brakelights(){
    if(Enable_Brakelights_Lights){
        foreach(Light L in BrakeLights){
            L.enabled = true;
        }
    }

    if(Enable_Brakelights_MeshRenderers){
        foreach(MeshRenderer BM in BrakeLights_MeshRenderers){
            BM.enabled = true;
        }
    }
}

public void Turn_Off_Brakelights(){
    if(Enable_Brakelights_Lights){
        foreach(Light L in BrakeLights){
            L.enabled = false;
        }
    }

    if(Enable_Brakelights_MeshRenderers){
        foreach(MeshRenderer BM in BrakeLights_MeshRenderers){
            BM.enabled = false;
        }
    }
}

//These are funtions for turning the headlights on & off (so I dont
copy/paste the same thing again and again)

```

```

public void Turn_On_Headlights() {
    if(Enable_Headlights_Lights){
        foreach(Light H in HeadLights){
            H.enabled = true;
        }
    }

    if(Enable_Headlights_MeshRenderers){
        foreach(MeshRenderer HM in HeadLights_MeshRenderers){
            HM.enabled = true;
        }
    }
}

public void Turn_Off_Headlights(){
    if(Enable_Headlights_Lights){
        foreach(Light H in HeadLights){
            H.enabled = false;
        }
    }

    if(Enable_Headlights_MeshRenderers){
        foreach(MeshRenderer HM in HeadLights_MeshRenderers){
            HM.enabled = false;
        }
    }
}

//Turn off/on reverse lights functions

public void Turn_Off_ReverseLights(){
    if(Enable_Reverselights_Lights){
        foreach(Light Rl in ReverseLights){
            Rl.enabled = false;
        }
    }

    if(Enable_Reverselights_MeshRenderers){
        foreach(MeshRenderer RM in ReverseLights_MeshRenderers){
            RM.enabled = false;
        }
    }
}

```

```

public void Turn_On_ReverseLights(){
    if(Enable_Reverselights_Lights){
        foreach(Light Rl in ReverseLights){
            Rl.enabled = true;
        }
    }

    if(Enable_Reverselights_MeshRenderers){
        foreach(MeshRenderer RM in ReverseLights_MeshRenderers){
            RM.enabled = true;
        }
    }
}

//Turn off car function
public void Turn_Off_Car(){
    Turn_Off_Headlights();
    Turn_Off_Brakelights();
    Turn_Off_ReverseLights();
    Car_Started = false;
}

//Function for setting wheel stiffness (not used, though it can aid
with oversteer)
public void Set_Stiffness(float Stiffness_Value){
    Wheel_forwardFriction.stiffness = Stiffness_Value;
    Wheel_backwardFriction.stiffness = Stiffness_Value;
}

//Boost function
public void Boost_Function(){
    rb.AddForce(Boost_Amount * transform.forward,
ForceMode.VelocityChange);

    if(Enable_Boost_particles){
        foreach(ParticleSystem P in Boost_particles){
            P.Play();
        }
    }
}
}

```

Car animation

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CarAnim : MonoBehaviour
{
    // idk if i have to use htis?
    [SerializeField] private Vector3 finalPosition;
    private Vector3 intialPosition;

    private void Awake()
    {
        intialPosition = transform.position;
    }

    private void Update()
    {
        transform.position = Vector3.Lerp(transform.position,
finalPosition, 0.1f);
    }

    private void OnDisable()
    {
        transform.position = intialPosition;
    }
}
```

Car selector

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class CarSelector : MonoBehaviour
{
    // note: make sure that methods work accros scenes
    [SerializeField] private Button previousButton;
```

```

[SerializeField] private Button nextButton;
[SerializeField] private Button selectButton; // Button to confirm
car selection

private int currentCar;

private void Awake()
{
    int savedCar = PlayerPrefs.GetInt("SelectedCar", 0); // Load
previous selection
    SelectCar(savedCar);
}

private void SelectCar(int _index)
{
    previousButton.interactable = (_index != 0);
    nextButton.interactable = (_index != transform.childCount - 1);

    for (int i = 0; i < transform.childCount; i++)
    {
        transform.GetChild(i).gameObject.SetActive(i == _index);
    }

    currentCar = _index;
}

public void ChangeCar(int _change)
{
    currentCar += _change;
    SelectCar(currentCar);
}

public void ConfirmCarSelection()
{
    PlayerPrefs.SetInt("SelectedCar", currentCar); // Save selected
car
    PlayerPrefs.Save();
    SceneManager.LoadScene("Map Selection"); // Load the Map
Selection Scene
}
}

```

Game loader

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
// Temp script
public class GameLoader : MonoBehaviour
{
    public Transform carParent; // Parent GameObject containing all
    car models

    private void Start()
    {
        int selectedCar = PlayerPrefs.GetInt("SelectedCar", 0); // Load
        selected car

        for (int i = 0; i < carParent.childCount; i++)
        {
            carParent.GetChild(i).gameObject.SetActive(i ==
            selectedCar);
        }
    }
}
```

Login register manager (dbms/mongodb)

```
using System;
using System.Threading.Tasks;
using UnityEngine;
using TMPro;
using MongoDB.Driver;
using MongoDB.Bson;
using UnityEngine.SceneManagement;
using System.Text.RegularExpressions;

public class LoginRegisterManager : MonoBehaviour
{
    [Header("input fields")]
    public TMP_InputField usernameInput;
    public TMP_InputField passwordInput;

    private MongoClient client;
```

```

private IMongoDatabase database;
private IMongoCollection<BsonDocument> userCollection;

void Start()
{
    // connect to mongodb (CHANGE PASSWORD)
    string connectionString =
    client = new MongoClient(connectionString);

    // connect to database (chk spelling)
    database = client.GetDatabase("loginL2D");

    // connect to collection(chk spelling)
    userCollection = database.GetCollection<BsonDocument>("users");
}

// password validation system
private bool IsPasswordStrong(string password)
{
    if (password.Length < 8)
        return false;
    if (!Regex.IsMatch(password, @"[A-Z]"))
        return false;
    if (!Regex.IsMatch(password, @"[a-z]"))
        return false;
    if (!Regex.IsMatch(password, @"[0-9]"))
        return false;
    if (!Regex.IsMatch(password, @"[\W_]"))
        return false;

    return true;
}

public async void OnRegisterClicked()
{
    // validation system response via debug log or console
    string username = usernameInput.text.Trim();
    string password = passwordInput.text;

    if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
    {
        Debug.LogWarning("username or password cannot be empty");
    }
}

```

```

        return;
    }

    if (!IsPasswordStrong(password))
    {
        Debug.LogWarning("password must be at least 8 characters
with uppercase, lowercase, number, and special symbol");
        return;
    }

    var existingUser = await
userCollection.Find(Builders<BsonDocument>.Filter.Eq("username",
username)).FirstOrDefaultAsync();

    if (existingUser != null)
    {
        Debug.LogWarning("user already exists");
        return;
    }

    var newUser = new BsonDocument
    {
        { "username", username },
        { "password", password }
    };

    await userCollection.InsertOneAsync(newUser);
    Debug.Log("user registered successfully");

    SceneManager.LoadScene("Main Menu");
}

public async void OnLoginClicked()
{
    string username = usernameInput.text.Trim();
    string password = passwordInput.text;

    var filter = Builders<BsonDocument>.Filter.And(
        Builders<BsonDocument>.Filter.Eq("username", username),
        Builders<BsonDocument>.Filter.Eq("password", password)
    );

```

```

        var user = await
userCollection.Find(filter).FirstOrDefaultAsync();

        if (user != null)
        {
            Debug.Log("login successful");
            SceneManager.LoadScene("Main Menu");
        }
        else
        {
            Debug.LogWarning("invalid username or password");
        }
    }
}

```

Main menu scene manager

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenuManager : MonoBehaviour
{
    // load Car Selection scene (stuck with scene name bc the register
was getting too long)
    public void OnPlayClicked()
    {
        SceneManager.LoadScene("CarSelection");
    }

    // load Test Menu scene
    public void OnTestClicked()
    {
        SceneManager.LoadScene("Test Menu");
    }

    // load Learn Menu scene
    public void OnLearnClicked()
    {
        SceneManager.LoadScene("learn Menu");
    }
}

```

Map selector

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MapSelector : MonoBehaviour
{
    public void SelectMap(string mapSceneName)
    {
        PlayerPrefs.SetString("SelectedMap", mapSceneName); // Save
selected map name
        PlayerPrefs.Save();
        SceneManager.LoadScene(mapSceneName); // Load the selected map
scene
    }
}
```

Minimap for car and map

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class Minimap_Cam : MonoBehaviour
{
    [Header("Objects Settings")]
    public Transform Target; //Target (the car)
    public Camera Camera; //The camera (the one this is assigned to)

    [Header("Car_Icon")]
    public Transform Icon; //The icon of the car
    public Vector3 Icon_Rotation_Offset; //The icon rotation offset
    public float Height_Offset; //How high the camera is

    [Header("Settings")]
    public Vector3 Offset_Pos; //The position offset
```

```

public float Cam_Size; //The camera size

void Update(){
    //Setting Camera Position To Target's Position + Offset
    transform.position = Target.position + Offset_Pos;

    //Setting Camera Size to Cam_Size
    Camera.orthographicSize = Cam_Size;

    //Setting the car's rotation and position to the Car Icon
    Icon.rotation = Target.rotation *
Quaternion.Euler(Icon_Rotation_Offset);
    Icon.position = Target.position + new Vector3(0, Height_Offset,
0);
}
}

```

Scene Switcher

```

using UnityEngine;
using UnityEngine.SceneManagement;

public class SStart : MonoBehaviour
{
    public void Log()
    {
        SceneManager.LoadScene("Login_Menu");
    }

    public void Reg()
    {
        SceneManager.LoadScene("Register_Menu");
    }
}

```

Speedometer things

```

using System.Collections;
using System.Collections.Generic;

```

```
using System.Linq;
using UnityEngine;
using TMPro;

public class Speedometer_Controller : MonoBehaviour
{
    //Public Vars
    [Header("General Settings")]
    public Rigidbody Car;

    public float Maximum_Speed = 0.0f;

    public float Minimum_Speed_Arrow_Angle;
    public float Maximum_Speed_Arrow_Angle;

    public bool Speed_In_KPH;

    [Header("Gears Setting")]
    public List<int> Gear_Speeds;

    [Header("UI Settings")]
    public TextMeshProUGUI Speed_Text;
    public RectTransform Arrow;
    public TextMeshProUGUI Gear_Text;

    [Header("Audio Settings")]

    public bool Use_Audio_Settings;
    public AudioSource Gear_Shift_Up;
    public AudioSource Gear_Shift_Down;

    [Header("Script Reference(s)")]
    public Car_Controller car_controller_script;

    [Header("Debug")]
    public string Current_Gear;
    public int Current_Gear_num;
    public bool is_reversing;
    public bool is_braking;
    public bool speed_is_zero;

    //Private Vars
    private float speed = 0.0f;
```

```

private int Speed_To_Display;

private void Start(){
    //Set Gears to 0:
    Current_Gear = "0";
    Current_Gear_num = 0;
}

private void Update()
{
    //Show speed in KPH
    if(Speed_In_KPH && !car_controller_script.Is_Flying()){
        speed = Car.velocity.magnitude * 3.6f;
    }

    //Show speed in MPH
    if(!Speed_In_KPH && !car_controller_script.Is_Flying()){
        speed = Car.velocity.magnitude * 2.237f;
    }

    //If speed_text var is assigned
    if (Speed_Text != null){
        //Change text of the Text var to the speed
        Speed_To_Display = (int)speed;
        Speed_Text.SetText(Speed_To_Display.ToString());
    }

    //if arrow rect var is assigned
    if (Arrow != null){
        //Rotate the arrow
        Arrow.localEulerAngles = new Vector3(0, 0,
Mathf.Lerp(Minimum_Speed_Arrow_Angle, Maximum_Speed_Arrow_Angle, speed
/ Maximum_Speed));
    }

    //do this if the speed is NOT equal to 0
    if(!speed_is_zero){
        //Setting the gear text to the current gear
        Gear_Text.SetText(Current_Gear);
    }
}

private void FixedUpdate(){

```

```

        //Changing Gears

        //Check is car isn't braking or reversing
        if(!is_braking && !is_reversing){
            //When the current speed is more than the speed of the
current gear
            if(Gear_Speeds[Current_Gear_num] < speed &&
Current_Gear_num != Gear_Speeds.Count &&
!car_controller_script.Is_Flying()){
                Gear_Up();
                Current_Gear = (Current_Gear_num + 1).ToString();
            }

            //When current speed is less than the speed of the current
gear
            if(Gear_Speeds[Current_Gear_num] > speed &&
Current_Gear_num != 0 && !car_controller_script.Is_Flying()){
                Gear_Down();
                Current_Gear = Current_Gear_num.ToString();
            }

            //When the last gear speed is the same as the current speed
            //This basically limits the speed of the car, so the car
has a max speed and does not exceed it
            if(Gear_Speeds.Last() <= speed){
                Car.velocity = Vector3.ClampMagnitude(Car.velocity,
Maximum_Speed);

                if(Gear_Speeds.Last() < speed){
                    Current_Gear = "N/A";
                }
            }

            //When speed is equal to 0
            if(speed == 0){
                speed_is_zero = true;
            }

            //When speed is not equal to 0
            if(speed != 0){
                speed_is_zero = false;
            }
        }

```

```

        //When car is reversing
        if(Input.GetAxis("Vertical") < 0f){
            is_reversing = true;
        }

        //When Car is Braking
        if(Input.GetKey(KeyCode.Space)){
            is_braking = true;
        }

        //When Car is not braking
        if(!Input.GetKey(KeyCode.Space)){
            is_braking = false;
        }

        //When car is moving forward
        if(Input.GetAxis("Vertical") > 0f){
            is_reversing = false;
        }

        //Check if car is braking or reversing and set the gear
        accordingly
        if(is_reversing){
            //Change gear to "R"
            Current_Gear = "R";
        }

        if(is_braking){
            //Set the Current Gear to B
            Current_Gear = "B";
        }

        //Change gear to zero if the bool is true
        if(speed_is_zero){
            Current_Gear = "0"; // very weird how 1st gear and nutral
            became gear 0? but it works
        }
    }

    //Methods for Gear Up & Down
    public void Gear_Up(){
        Current_Gear_num++;
    }

```

```

        if(Use_Audio_Settings){
            //Play gear shift up effect
            Gear_Shift_Up.Play();
        }
    }

    public void Gear_Down(){
        Current_Gear_num--;

        if(Use_Audio_Settings){
            //Play gear shift down audio
            Gear_Shift_Down.Play();
        }
    }
}

```

Test manager

```

using System.Collections.Generic;
using UnityEngine;
using TMPro;
using UnityEngine.UI;

public class TestManager : MonoBehaviour
{
    // array of questions per topic AND/OR difficulty based
    public TMP_Text questionText;
    public TMP_InputField answerInput;
    public TMP_Dropdown topicDropdown;

    private string selectedTopic;
    private int difficultyLevel = 0;

    private Dictionary<string, List<string[]>> topicQuestions = new
Dictionary<string, List<string[]>>();
    private Dictionary<string, List<string[]>> topicAnswers = new
Dictionary<string, List<string[]>>();

    private int currentQuestionIndex = 0;

    void Start()
    {

```

```

        SetupQuestions();
        topicDropdown.onValueChanged.AddListener(delegate {
OnTopicSelected(); });
    }

    void SetupQuestions()
    // per topic 6 total questions
    {
        topicQuestions["Driving Laws"] = new List<string[]> {
            new string[] { "What is the legal alcohol limit in mg?",
"What age can you get a learner's license?" },
            new string[] { "What does a double yellow line mean?", "Max
speed limit in urban areas?" },
            new string[] { "When can you overtake on a single line?",
"Penalties for reckless driving?" }
        };

        topicAnswers["Driving Laws"] = new List<string[]> {
            new string[] { "80", "16" },
            new string[] { "no overtaking", "50" },
            new string[] { "when clear", "license suspension" }
        };

        topicQuestions["Road Signs"] = new List<string[]> {
            new string[] { "What does a red octagon mean?", "What does
a triangle mean?" },
            new string[] { "What does a circular blue sign mean?",
"What does a yellow diamond mean?" },
            new string[] { "Explain the difference between regulatory
and warning signs?", "When do you ignore signs?" }
        };

        topicAnswers["Road Signs"] = new List<string[]> {
            new string[] { "stop", "yield" },
            new string[] { "mandatory", "caution" },
            new string[] { "rules warning", "never" }
        };
    }

    public void OnTopicSelected()
    {
        selectedTopic =
topicDropdown.options[topicDropdown.value].text;

```

```

        difficultyLevel = 0;
        currentQuestionIndex = 0;
        ShowQuestion();
    }

    public void CheckAnswer()
    {
        string userAnswer = answerInput.text.Trim().ToLower();
        string correctAnswer =
topicAnswers[selectedTopic][difficultyLevel][currentQuestionIndex].ToLo
wer();

        if (userAnswer == correctAnswer)
        {
            difficultyLevel = Mathf.Min(difficultyLevel + 1,
topicQuestions[selectedTopic].Count - 1);
        }
        else
        {
            difficultyLevel = Mathf.Max(difficultyLevel - 1, 0);
        }

        currentQuestionIndex = (currentQuestionIndex + 1) %
topicQuestions[selectedTopic][difficultyLevel].Length;
        ShowQuestion();
    }

    void ShowQuestion()
    {
        string question =
topicQuestions[selectedTopic][difficultyLevel][currentQuestionIndex];
        questionText.text = question;
        answerInput.text = "";
    }
}

```

WHeel effects

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

```

```

public class Wheel_Effects : MonoBehaviour
{
    //Variables
    [Header("Tire Mark Trail Renderers")]
    public TrailRenderer[] Tire_Marks; //The Trail renderer(s) for the
tracks

    [Header("Tire Mark Particle Systems")]
    public bool Enable_Particle_System;
    public ParticleSystem[] Skid_Particles; //The particle system(s)
for the particles

    [Header("Tire Audio (Basically The Sound When Drifting)")]
    public bool Enable_Audio;
    public AudioSource Drift_Audio; //This would be like a tire screech
sound

    [Header("Script References")]
    public Car_Controller car_Controller;

    //Privat vars
    private bool Tire_Marks_Flag;
    private bool is_drifting;
    private bool Is_Flying;

    public void Start(){
        foreach(TrailRenderer T in Tire_Marks){
            T.emitting = false;
        }

        if(Enable_Particle_System){
            foreach(ParticleSystem P in Skid_Particles){
                P.Stop();
            }
        }
    }

    //Update function to check the drifting every frame
    void FixedUpdate(){
        Check_Drift();
        Is_Flying = car_Controller.Is_Flying();
    }
}

```

```

//Check if drifting or braking
public void Check_Drift(){
    if(Input.GetKey(KeyCode.Space) && !Is_Flying){
        StartEmitter();
        is_drifting = false;
    }

    else if (car_Controller.tempo != 0.5 && !Is_Flying){
        StartEmitter();
        is_drifting = true;
    }

    else{
        StopEmitter();
        is_drifting = false;
    }
}

//Start Renderring Trail
public void StartEmitter(){
    if(Tire_Marks_Flag) return;

    //Drift Trail
    foreach (TrailRenderer T in Tire_Marks)
    {
        T.emitting = true;
    }

    //Drift Particles
    if(Enable_Particle_System && is_drifting){
        foreach (ParticleSystem P in Skid_Particles)
        {
            P.Play();
        }
    }

    //Drift Sound
    if(Enable_Audio && is_drifting){
        if(!Drift_Audio.isPlaying){
            //Play the sound
            Drift_Audio.Play();
        }
    }
}

```

```

    }

    Tire_Marks_Flag = true;
}

//Stop Renderring Trail
public void StopEmitter(){
    if(!Tire_Marks_Flag) return;

    //Drift Trai
    foreach (TrailRenderer T in Tire_Marks)
    {
        T.emitting = false;
    }

    //Drift Particles
    if(Enable_Particle_System){
        foreach (ParticleSystem P in Skid_Particles)
        {
            P.Stop();
        }
    }

    //Drift Sound
    if(Enable_Audio){
        //Stop playing the sound
        Drift_Audio.Stop();
    }

    Tire_Marks_Flag = false;
}
} // note that tire skid is 2d
// side note, the sound overlaps and echoes due to the audio size
length and stuff, cant change that. dont ue it ig.

```

QUIZ MANAGER

```

using UnityEngine;
using TMPro;
using UnityEngine.UI;
using System.Collections.Generic;

```

```

// time to get quizical (/b \b = bold) + (\n = line brek \n\n = 2 line
space)
public class QuizController : MonoBehaviour
{
    [Header("UI Panels")]
    public GameObject topicSelectionPanel; // panel with 3 buttons:
Signs, Laws, Rules // *CAPITALIZATION* dont mess it up
    public GameObject quizPanel;          // main quiz UI
    public GameObject finishedPanel;      // final summary

    [Header("UI Elements (Quiz Panel)")]
    public TextMeshProUGUI questionField; // shows current question
    public TMP_InputField answerField;    // user's answer input
    public Button submitButton;           //submit answer for
verification NOT validation
    public Button nextButton;             // next question
    public TextMeshProUGUI feedbackText;  // shows correctness +
explanation
    public TextMeshProUGUI progressText;  // "Question X / Y"
    public TextMeshProUGUI scoreText;     // show running score

    [Header("Finished UI")]
    public TextMeshProUGUI finalScoreText;
    public TextMeshProUGUI finalSummaryText;

    // Internal state
    private string currentTopic = "";
    private int difficultyIndex = 0; // 0 --> 4 for progressive
difficulty *success criteria
    private int questionIndex = 0;     // index inside current
difficulty array
    private int score = 0;
    private int totalQuestions = 0;

    // Questions: topic -> difficulty -> array of questions
    // Each question has a matching answer and (optional) explanation
    private Dictionary<string, string[][]> questions;
    private Dictionary<string, string[][]> answers;
    private Dictionary<string, string[][]> explanations;

    void Awake()
    {
        InitializeDatabase();
    }
}

```

```

// UI initial state / STARTUP
topicSelectionPanel.SetActive(true);
quizPanel.SetActive(false);
finishedPanel.SetActive(false);

submitButton.onClick.AddListener(OnSubmitPressed);
nextButton.onClick.AddListener(OnNextPressed);
nextButton.gameObject.SetActive(false); // hidden until after
submit
}

void InitializeDatabase()
{
    // Create dictionaries and populate with 5 difficulty arrays
each. (for now ig)
    questions = new Dictionary<string, string[][]>();
    answers = new Dictionary<string, string[][]>();
    explanations = new Dictionary<string, string[][]>();

    // --- SIGNS ----
    questions["Signs"] = new string[][]
    {
        // up to diff 4 should all be correct for a pass
        // difficulty 0 (easy)
        new string[]
        {
            "What does a red octagon sign mean?",
            "What does a triangular sign with a red border usually
indicate?",
            "Which sign means 'No entry'?"
        },
        // difficulty 1
        new string[]
        {
            "What does a circular blue sign with a white arrow
indicate?",
            "What does a yellow diamond sign typically mean in some
countries?",
            "What sign warns of a pedestrian crossing?"
        },
        // difficulty 2
        new string[]
        {

```

```

        "What does a circular sign with a red border and a
number mean?",
        "What does a red circle with a white horizontal bar
indicate?",
        "What sign shows that you must give way?"
    },
    // difficulty 3 (med)
    new string[]
    {
        "What is the meaning of a sign showing a truck with a
red line through it?",
        "Describe the meaning of a white sign with black
diagonal stripes.",
        "What does a sign with a bicycle inside a red circle
mean?"
    },
    // difficulty 4 (hard)
    new string[]
    {
        "What does a blue rectangular sign with white letters
'M' usually indicate in some countries?",
        "What is the purpose of an 'End of all restrictions'
round sign?",
        "Explain the meaning of a flashing amber traffic light
at a pedestrian crossing."
    }
};

// organized as q --> a or a-->q
answers["Signs"] = new string[][]
{
    new string[] { "stop", "warning", "no entry" },
    new string[] { "mandatory direction", "warning",
"pedestrian crossing" },
    new string[] { "speed limit", "no entry", "give way" },
    new string[] { "no heavy goods vehicles / no trucks", "end
of restrictions / national speed limit", "no bicycles" },
    new string[] { "motorway", "end of restrictions", "proceed
with caution; give way to pedestrians" }
};

explanations["Signs"] = new string[][]
{

```

```

        new string[] { "A red octagon is universally 'Stop'.",
        "Triangular red-border signs are warnings.", "A red circle with white
dash or a red circle with white rectangle often means 'No entry'." },
        new string[] { "Blue circle with arrow = mandatory
direction.", "Yellow diamond often indicates priority or warning
depending on region.", "Marked sign or zebra symbols warn of
crossings." },
        new string[] { "Numbers inside red circle indicate speed
limits.", "Red circle with white horizontal bar = No entry.", "Give way
signs are inverted triangle or 'Yield'." },
        new string[] { "Truck with red line = vehicles of that type
prohibited.", "White with black stripes often signifies end of previous
restrictions.", "Bicycle in red circle = bicycles prohibited." },
        new string[] { "M = motorway in many countries.", "End of
all restrictions clears prior speed and passing rules.", "Flashing
amber = stop when pedestrians are present or be prepared to stop; give
way to pedestrians." }
    };

    // -- LAWS --
    questions["Laws"] = new string[][]
    {
        new string[] {
            "What is the typical UK national speed limit for single
carriageway roads (mph)?",
            "Should you use a mobile phone while driving without
hands-free? (yes/no)",
            "Who has priority at a roundabout?"
        },
        new string[] {
            "At what blood alcohol limit is it illegal to drive in
many countries? (answer with 'low'/'very low'/'0.08' etc)",
            "When approaching a zebra crossing with pedestrians
waiting, what should you do?",
            "Is it legal to overtake on the left in general?
(yes/no)"
        },
        new string[] {
            "When must you use dipped headlights?",
            "What is the minimum following distance rule (in
seconds) in good conditions?",
            "If an emergency vehicle approaches with lights on,
what should you do?"
        }
    }

```

```

        },
        new string[] {
            "What documents must you carry when driving (three items)?",
            "What is the rule for child car seats up to 12 years or 135cm in many countries?",
            "What is 'constructive driving' offence commonly called (short phrase)?"
        },
        new string[] {
            "Explain what 'drink-drive' laws intend to prevent (short).",
            "Describe the consequences of refusing a breath test in many jurisdictions.",
            "When can a provisional license holder carry passengers (brief)?"
        }
    };

    answers["Laws"] = new string[][]
    {
        new string[] { "60", "no", "vehicles on roundabout/traffic on roundabout" },
        new string[] { "0.08", "stop and let them cross/slow and give way", "no" },
        new string[] { "in poor visibility/at night when required", "2", "pull over and let them pass / give way" },
        new string[] { "license insurance v5 (or registration) or id", "use appropriate child restraint/booster", "dangerous driving" },
        new string[] { "prevent impaired driving / reduce accidents", "penalties, fines, licence loss or arrest", "when supervised/instructor (varies by country)" }
    };

    explanations["Laws"] = new string[][]
    {
        new string[] { "UK single carriageway national limit is 60mph for cars unless signed otherwise.", "Using handheld phone is illegal and distracts the driver.", "Traffic already on the roundabout usually has priority." },
        new string[] { "Common legal limit is 0.08% BAC in some countries (varies).", "Give way to pedestrians; stop if they are"

```

```

crossing.", "Overtaking on left is generally not allowed except in
special cases." },

    new string[] { "Dipped headlights at night or poor
visibility.", "A 2 second rule is a common minimum in good conditions;
increase when wet.", "Move aside safely and allow emergency vehicle
through." },

    new string[] { "Carry driving license, insurance, and
vehicle registration/document as a minimum (depends on country).",
"Children should use appropriate child restraint/booster until 12 years
or 135cm in many regions.", "Dangerous driving is the common legal term
for very poor driving." },

    new string[] { "They reduce the risk of accidents due to
intoxication and protect others on the road.", "Refusal often has
severe consequences: licence suspension, fines, or arrest (varies).",
"Often only with an approved supervisor or not allowed – depends on the
country's rules." }

};

// --- RULES ---
questions["Rules"] = new string[][]
{
    new string[] {
        "Which side of the road do you drive on in the UK?",
        "When do you indicate? (short)",
        "What does double yellow line at the kerb usually
mean?"

    },
    new string[] {
        "At an uncontrolled junction, who should you give way
to?",

        "What is the purpose of a box junction?",
        "When can you use the hard shoulder in emergencies?"

    },
    new string[] {
        "What does lane discipline mean (brief)?",
        "When merging, who should give way?",
        "What should you check before opening a car door on the
traffic side?"

    },
    new string[] {
        "Explain purpose of chevrons on a motorway slip road.",
        "When parking uphill with no curb, which way to
steer?",

```

```

        "What is mirror-signal-manoeuve sequence?"
    },
    new string[] {
        "Describe what 'undertaking' means and if it is
allowed.",
        "Explain 'staggered junction' briefly.",
        "What is the correct procedure when an obstruction
blocks the lane?"
    }
};

answers["Rules"] = new string[][]
{
    new string[] { "left", "to signal intention / when turning
or changing lane", "no parking" },
    new string[] { "traffic from right (or main road)",
"prevent blocking junctions", "only in breakdowns/emergency" },
    new string[] { "keeping to your lane and only changing when
safe", "the merging traffic should give way unless signed", "look and
check mirrors and blind spot" },
    new string[] { "guide/speed control on slip road / keep
safe approach", "turn wheels away from road (or into verge) - depends
on direction; for UK steer away from kerb downhill = towards left?
(common guidance: uphill turn towards curb if there is one)", "check
mirrors, signal intent, then carry out manoeuvre" },
    new string[] { "undertaking = passing on the inside;
usually not allowed except in slow queue", "two staggered T junctions
arranged to avoid direct cross traffic", "slow, signal, and rejoin when
safe; follow diversions" }
};

explanations["Rules"] = new string[][]
{
    new string[] { "UK drives on the left. Indicate to show
turning/changing lanes. Double yellow lines usually mean no parking at
any time (subject to local signs).", "At uncontrolled junctions you
usually give way to traffic from the right in most countries (UK:
priority rules). Box junctions keep junctions clear. Hard shoulder is
for emergencies only.", "Lane discipline = keep lane unless
overtaking/turning; merging rules prevent collisions; check blind spots
to avoid harming cyclists." }
};

```

```

        // organize in future properly
        //questions from gov.uk
    }

    // Button hook: called by the 3 topic buttons with parameter
    "Signs", "Laws", or "Rules" I LOVE THIS
    public void SelectTopic(string topicName)
    {
        if (!questions.ContainsKey(topicName))
        {
            Debug.LogError("Topic not found: " + topicName); //caps are
important
            return;
        }

        currentTopic = topicName;
        topicSelectionPanel.SetActive(false);
        quizPanel.SetActive(true);
        finishedPanel.SetActive(false);

        // reset state
        difficultyIndex = 0;
        questionIndex = 0;
        score = 0;
        totalQuestions = CountTotalQuestionsForTopic(topicName);

        UpdateScoreUI();
        ShowCurrentQuestion();
    }

    int CountTotalQuestionsForTopic(string topic)
    {
        int total = 0;
        var arr = questions[topic];
        for (int i = 0; i < arr.Length; i++)
            total += arr[i].Length;
        return total;
    }

    void ShowCurrentQuestion()
    {
        // Defensive checks
        if (difficultyIndex >= 5)

```

```

    {
        FinishQuiz();
        return;
    }

    string[] difficultyArray =
questions[currentTopic][difficultyIndex];
    if (questionIndex >= difficultyArray.Length)
    {
        // move to next difficulty
        difficultyIndex++;
        questionIndex = 0;
        ShowCurrentQuestion();
        return;
    }

    string q = difficultyArray[questionIndex];
    questionField.text = $"(Topic: {currentTopic}) [Difficulty
{difficultyIndex + 1}/5]\n\n{q}";
    answerField.text = "";
    feedbackText.text = "";
    submitButton.interactable = true;
    nextButton.gameObject.SetActive(false);
    UpdateProgressUI();
}

void UpdateProgressUI()
{
    int askedSoFar = 0;
    for (int d = 0; d < difficultyIndex; d++)
        askedSoFar += questions[currentTopic][d].Length;
    askedSoFar += questionIndex + 1;
    progressText.text = $"Question {askedSoFar} /
{totalQuestions}";
}

void UpdateScoreUI()
{
    scoreText.text = $"Score: {score}";
}

public void OnSubmitPressed()
{

```

```

submitButton.interactable = false;
string userAnswer = answerField.text.Trim().ToLower();
string correct = "";

// fetch correct answer (if available)
if (answers.ContainsKey(currentTopic) && difficultyIndex <
answers[currentTopic].Length)
{
    var ansArray = answers[currentTopic][difficultyIndex];
    if (questionIndex < ansArray.Length)
        correct = ansArray[questionIndex].ToLower();
}

bool isCorrect = false;
if (!string.IsNullOrEmpty(correct))
{
    // simple matching: contains or equals (to allow slight
variations)
    if (userAnswer == correct || userAnswer.Contains(correct)
|| correct.Contains(userAnswer))
        isCorrect = true;
}
else
{
    // If no answer stored, mark as false but show message
feedbackText.text = "No model answer available for this
question.";
}

if (isCorrect)
{
    score++;
    feedbackText.text = "<b>Correct!</b>\n\n";
}
else
{
    feedbackText.text = "<b>Incorrect.</b>\n\n";
    // show expected short answer if present (helpful)
    if (!string.IsNullOrEmpty(correct))
        feedbackText.text += $"Expected: {correct}\n\n";
}

// add explanation if available

```

```

        if (explanations.ContainsKey(currentTopic)
            && difficultyIndex < explanations[currentTopic].Length
            && questionIndex <
explanations[currentTopic][difficultyIndex].Length)
        {
            feedbackText.text += "Explanation: " +
explanations[currentTopic][difficultyIndex][questionIndex];
        }

        nextButton.gameObject.SetActive(true);
        UpdateScoreUI();
    }

    public void OnNextPressed()
    {
        // move to next question within difficulty or next difficulty
        questionIndex++;
        // if passed end-of-difficulty, ShowCurrentQuestion handles
increment
        ShowCurrentQuestion();
    }

    void FinishQuiz()
    {
        quizPanel.SetActive(false);
        finishedPanel.SetActive(true);
        finalScoreText.text = $"Final score: {score} /
{totalQuestions}";
        finalSummaryText.text = $"You completed the {currentTopic}
test.\nCorrect: {score}\nIncorrect: {totalQuestions - score}";
    }

    // optional: hook this to a "Retry" button in finishedPanel
    public void RetryTopic()
    {
        topicSelectionPanel.SetActive(false);
        finishedPanel.SetActive(false);
        quizPanel.SetActive(true);
        difficultyIndex = 0;
        questionIndex = 0;
        score = 0;
        totalQuestions = CountTotalQuestionsForTopic(currentTopic);
        UpdateScoreUI();
    }

```

```
        ShowCurrentQuestion();
    }

    // optional: back to topic selection
    public void BackToTopics()
    {
        topicSelectionPanel.SetActive(true);
        quizPanel.SetActive(false);
        finishedPanel.SetActive(false);
    }
}
```